Date :- 10/10/26 React Concept

```
setTimeOut ( () => {

} , 1000) ;
```

Runtime Enviroment



Inversion of Control

- Promises ✓
- Async await ✓

Callback Hell

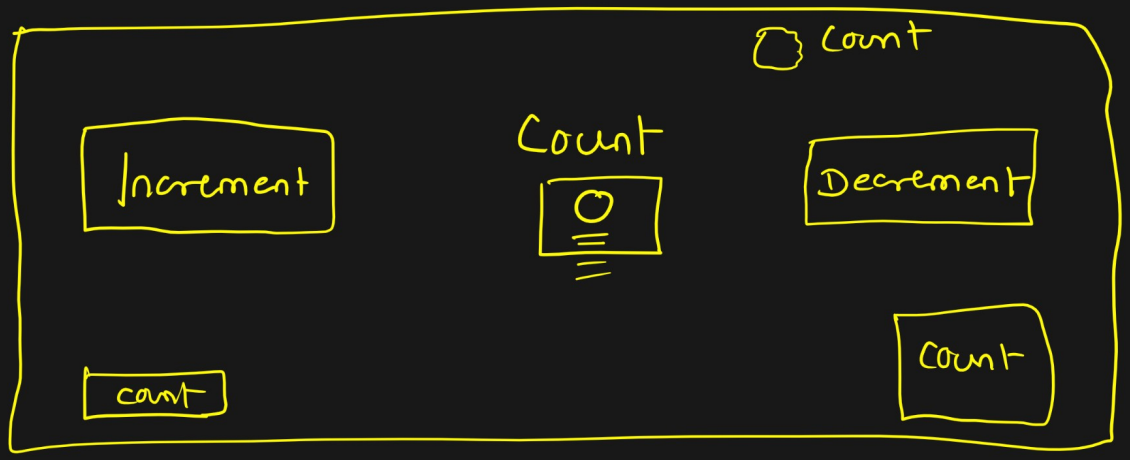
```
() => {
    () => {
        () => {
        }
    }
}
```

React Redux Toolkit

State Management :-

data

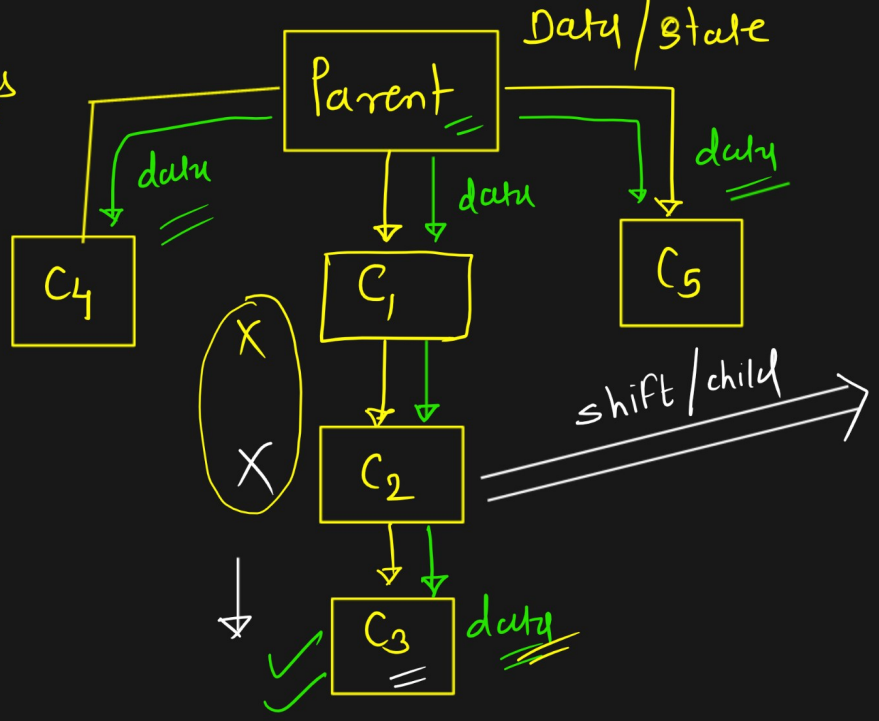
useState



Props Drilling issues

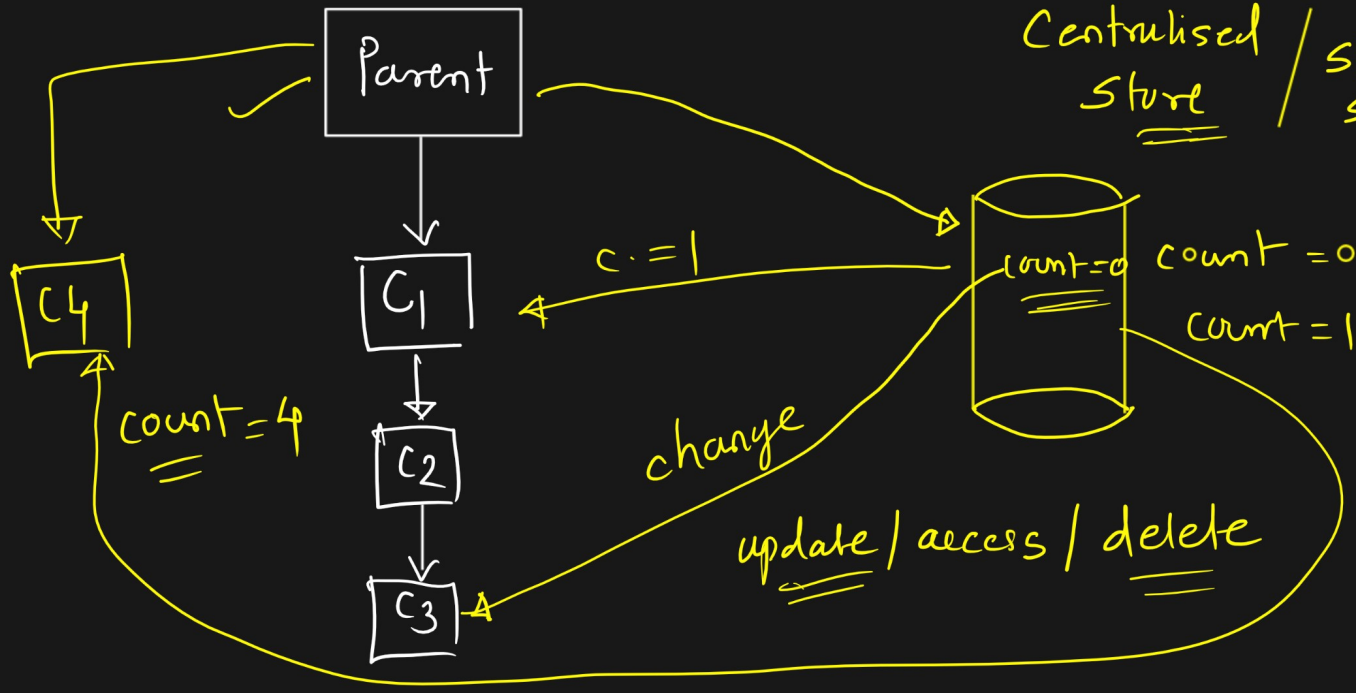
OR

Use Context

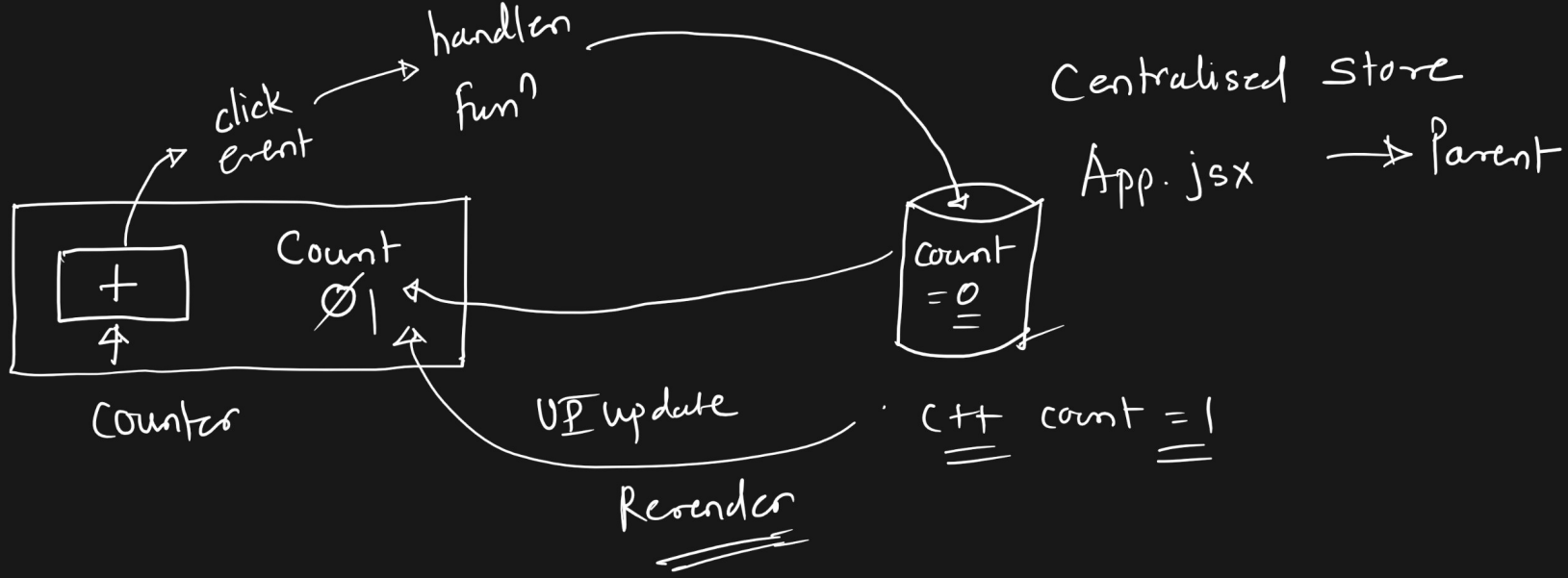


Structure / Tree

Distributed


Parent
Centralised Store / Single source of Truth
C4
C1
C2
C3
c. = 1
count = 4
change
count = 0
count = 1
update / access / delete

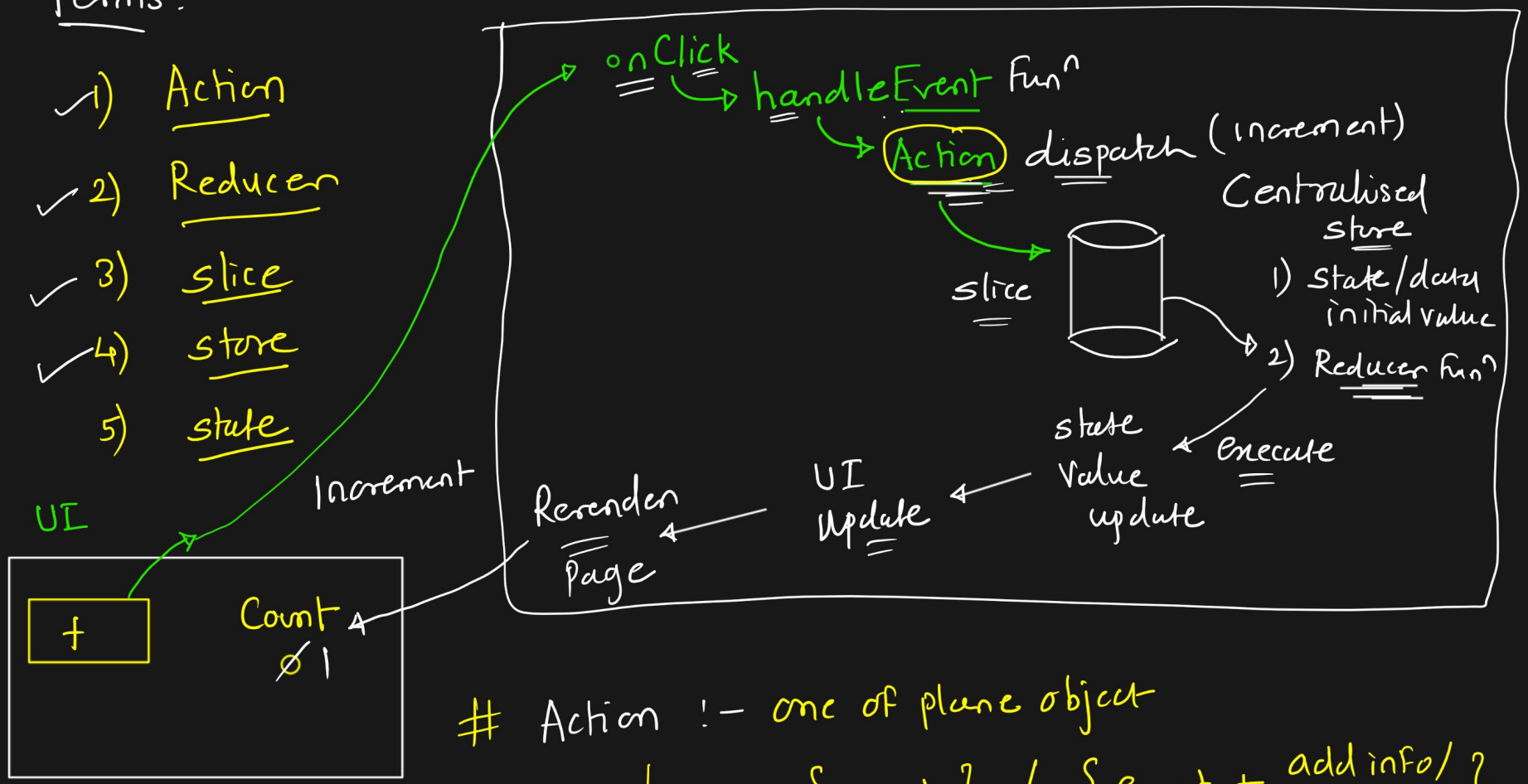
e.g.


handler fun?
click event
Count
0 1
+
Counter
count = 0
UI update
C++ count = 1
Rerender
Centralised Store
App.jsx → Parent

React Redux Toolkit :-

Terms :-

1) Action
2) Reducer
3) slice
4) store
5) state



Action :- one of plane object

{ event } / { event + add info/ payload }

event

e.g. value = 5 / = 4

Reducer :- Fun^ ⇒ that update the state value

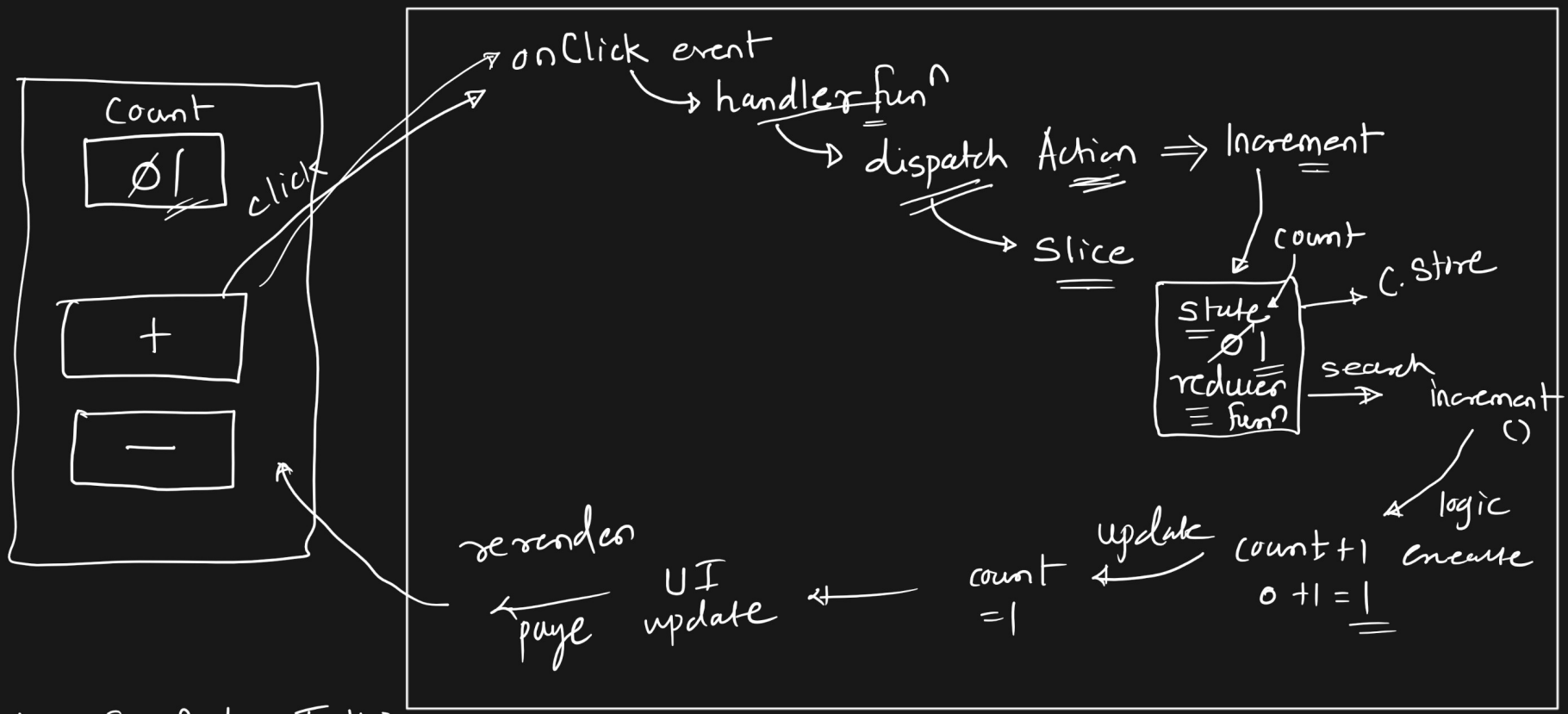
slice :- Appln → features → state → manage by slice

- auth :- login / logout ⇒ actions ⇒ user data ⇒ slice
- counter :- increm / decrement ⇒ actions ⇒ count ⇒ slice
- card :- add / remove / update ⇒ actions

slice → initial value of state ✓
slice → action → Reducer fun ✓
Reducer fun → action performed / defined
Reducer fun → logic to update the state value

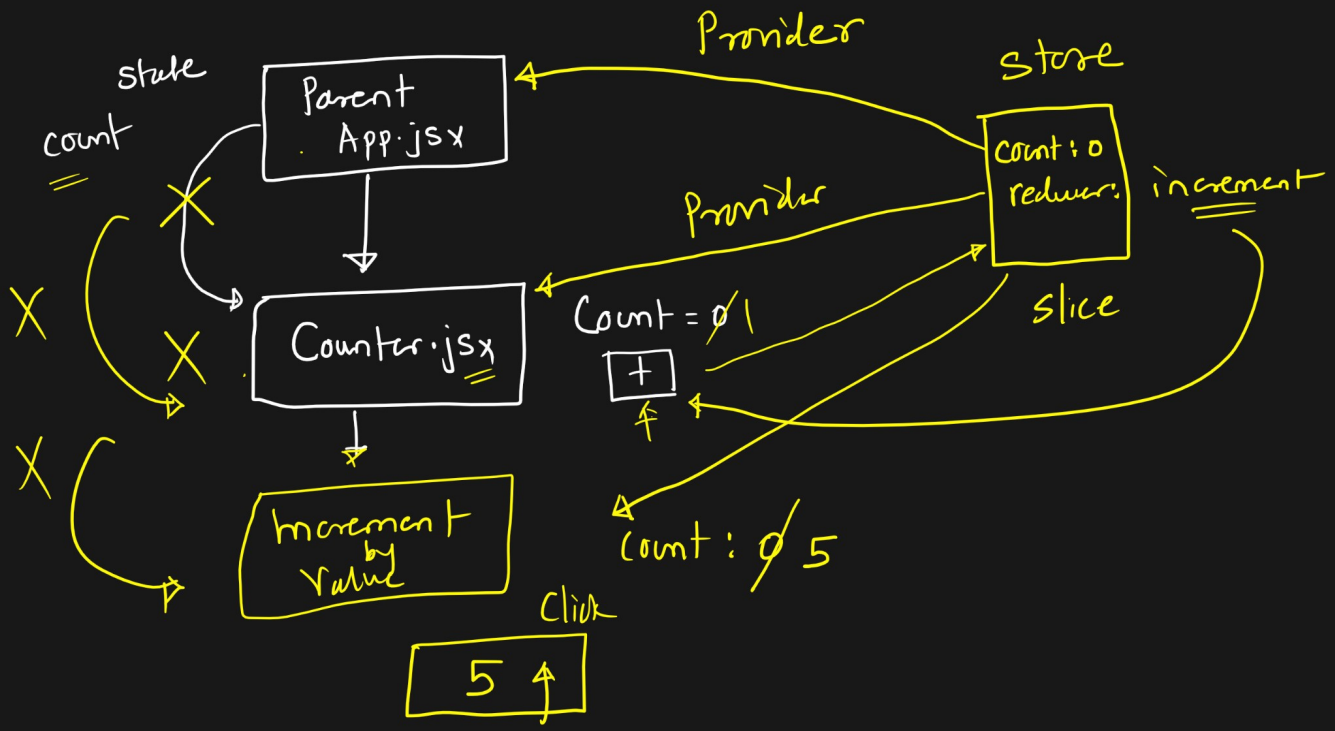
store :- data / state ⇒ single source of truth

state :- info / data



Steps for Redux Toolkit Appln

1) Install Redux Toolkit and React-Redux
2) Create a Redux Store
3) Provide the Redux Store to React
4) Create a Redux State Slice
5) Add Slice Reducers to the Store
6) Use Redux State and Actions in React Components


Provider
store
state
Parent
App.jsx
count
count : 0
reducer
increment
Provider
Counter.jsx
Count = 0 1
+
slice
Increment
by
value
Count : 0 5
Click
5